ANÁLISE E OTIMIZAÇÃO DE CONSULTAS SQL COM BASE EM LOGS DE EXECUÇÃO EM BANCOS DE DADOS RELACIONAIS

Arthur Meira[1], Marcel Campos Inocencio[2]

Resumo: Consultas SQL lentas podem gerar gargalos e comprometer o desempenho de sistemas que utilizam bancos de dados relacionais. Este trabalho teve como objetivo desenvolver uma ferramenta para auxiliar na identificação e na análise preventiva de consultas com possíveis problemas de desempenho. Para avaliar a solução, foi criado um banco de dados experimental com informações fictícias de clientes, produtos, pedidos, itens e pagamentos, totalizando 4.648.714 registros. Em seguida, foi executado um conjunto de consultas com diferentes características e frequências de utilização. As informações sobre essas execuções foram coletadas no PostgreSQL e combinadas com a análise dos planos de execução. A ferramenta desenvolvida identifica sinais como varreduras sequenciais em tabelas grandes, funções aplicadas em filtros, respostas com muitas linhas e diferenças entre a quantidade estimada e a quantidade histórica de registros. Os resultados permitiram localizar consultas com elevado tempo de execução e avaliar possíveis alterações em sua escrita. Em um dos experimentos, a substituição de uma função aplicada sobre uma coluna indexada permitiu que o banco utilizasse o índice existente. Nas condições avaliadas, essa alteração reduziu o tempo médio de execução no servidor de 26,33 para 0,0303 milissegundo, mantendo o mesmo resultado. Também foi desenvolvida uma interface web para apresentar as consultas, os planos, os sinais encontrados e as sugestões de melhoria. Concluiu-se que a combinação de estatísticas históricas, planos de execução e regras explicáveis pode auxiliar na investigação de consultas lentas, desde que as alterações sugeridas sejam confirmadas por testes de equivalência e desempenho.

Palavras-chave: bancos de dados relacionais; consultas SQL; otimização de consultas; estatísticas de execução; planos de execução.

ABSTRACT: Slow SQL queries can create bottlenecks and affect the perfor-

[1]Curso de Ciência da Computação, Universidade do Extremo Sul Catarinense (Unesc), Criciúma - Santa Catarina - Brasil. arthurmeiraprog@unesc.net

[2]Orientador, Curso de Ciência da Computação, Universidade do Extremo Sul Catarinense (Unesc), Criciúma - Santa Catarina - Brasil. marcel.inocencio@gmail.com

mance of systems that use relational databases. This study aimed to develop a tool to assist in the identification and preventive analysis of queries with possible performance problems. To evaluate the solution, an experimental database containing fictional data about customers, products, orders, order items, and payments was created, totaling 4,648,714 records. A set of queries with different characteristics and execution frequencies was then performed. Information about these executions was collected from PostgreSQL and combined with the analysis of execution plans. The developed tool identifies warning signs such as sequential scans on large tables, functions applied to filters, queries that return a large number of rows, and differences between the estimated and historical number of records. The results made it possible to identify queries with high execution times and evaluate possible changes to their SQL statements. In one experiment, removing a function applied to an indexed column allowed the database to use the existing index. Under the evaluated conditions, this change reduced the mean server execution time from 26.33 to 0.0303 milliseconds while preserving the same result. A web interface was also developed to present the queries, execution plans, detected warning signs, and improvement suggestions. It was concluded that combining historical statistics, execution plans, and explainable rules can support the investigation of slow queries, provided that the suggested changes are confirmed through equivalence and performance tests.

Keywords: relational databases; SQL queries; query optimization; execution statistics; execution plans.

1 INTRODUÇÃO

A linguagem SQL (*Structured Query Language*) é utilizada para consultar e manipular informações armazenadas em bancos de dados relacionais. Ao escrever uma consulta, o desenvolvedor informa quais dados deseja obter, enquanto o sistema gerenciador de banco de dados (SGBD) determina como essa busca será realizada. Uma mesma consulta pode ser executada de diferentes formas, dependendo da quantidade de dados, dos índices disponíveis, dos filtros aplicados e das operações necessárias para produzir o resultado (LAN; BAO; PENG, 2021).

A escolha da forma de execução é realizada pelo otimizador de consultas. Esse componente analisa diferentes possibilidades e seleciona um plano de execução. O plano apresenta as operações que o banco pretende realizar, como a leitura de tabelas, a aplicação de filtros, as junções,

os agrupamentos e as ordenações. Mesmo quando duas estratégias retornam os mesmos dados, elas podem apresentar diferenças significativas no uso de recursos e no tempo necessário para concluir a consulta.

De acordo com Lan, Bao e Peng (2021), a escolha de um plano depende principalmente da estimativa da quantidade de registros, do cálculo do custo das operações e da análise das alternativas disponíveis. A estimativa da quantidade de registros, também chamada de estimativa de cardinalidade, indica quantas linhas cada etapa do plano deve produzir. Quando essa previsão se distancia da quantidade real, o banco pode escolher uma estratégia menos adequada.

As características dos dados também influenciam esse processo. Kossmann, Papenbrock e Naumann (2022) discutem como restrições de unicidade e dependências entre atributos podem apoiar a otimização. Uma coluna sem valores repetidos, por exemplo, fornece uma informação diferente daquela que permite várias ocorrências do mesmo valor. Conhecer essas propriedades pode permitir simplificar operações, desde que as condições necessárias sejam válidas no conjunto analisado. Uma alteração baseada em uma suposição incorreta pode modificar o resultado esperado da consulta.

Além da análise do plano, é necessário observar o comportamento das consultas ao longo do uso do banco de dados. Uma consulta lenta executada poucas vezes possui um impacto diferente de uma consulta mais rápida que se repete milhares de vezes. Estatísticas como quantidade de chamadas, tempo total, tempo médio e número de linhas retornadas ajudam a identificar quais consultas merecem ser investigadas primeiro.

A análise de desempenho geralmente exige conhecimento técnico para interpretar essas informações. Uma varredura sequencial, por exemplo, não representa necessariamente um problema, pois pode ser adequada quando a consulta retorna grande parte de uma tabela. Da mesma forma, a criação de um índice não garante melhoria e pode aumentar o custo das operações de inserção e atualização. Por isso, as recomendações precisam ser apresentadas como possibilidades que devem ser verificadas por meio de testes.

A pesquisa se justifica pela necessidade de identificar e reduzir gargalos provocados por consultas SQL lentas. Esses gargalos podem aumentar o tempo de resposta das aplicações, elevar o consumo de recursos do servidor e prejudicar vários usuários ao mesmo tempo. Sem um processo de análise, alterações podem ser aplicadas sem evidências sufi-

cientes ou consultas de baixo impacto podem receber prioridade enquanto problemas mais relevantes permanecem sem investigação.

Diante desse contexto, esta pesquisa busca responder à seguinte questão: como combinar estatísticas históricas e planos de execução para identificar consultas SQL potencialmente problemáticas, explicar os sinais encontrados e apoiar a validação de melhorias em bancos de dados relacionais?

O objetivo geral é desenvolver e avaliar um processo automatizado de análise preventiva de consultas SQL baseado em estatísticas históricas e planos de execução. Os objetivos específicos são construir uma base experimental reprodutível, executar consultas com diferentes características, coletar e classificar suas métricas, identificar possíveis problemas nos planos, comparar consultas originais e alternativas, validar o efeito das alterações e apresentar as informações em uma interface visual.

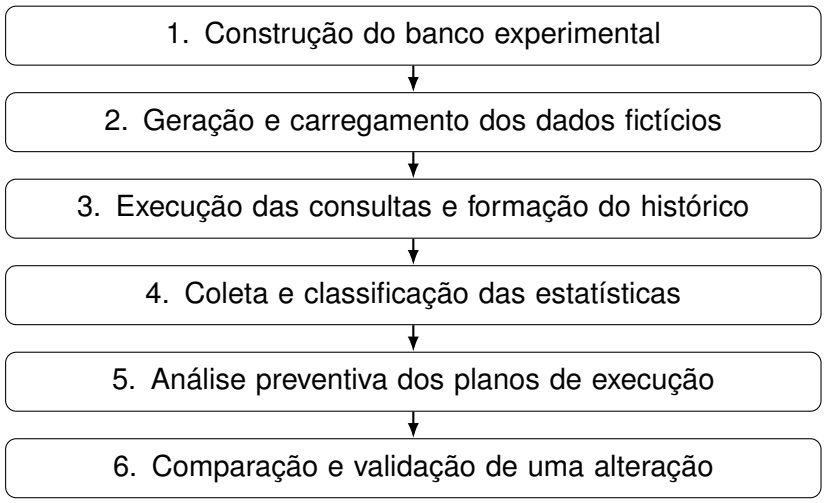
A principal contribuição deste trabalho está na integração dessas etapas em uma ferramenta de apoio. A solução não aplica alterações automaticamente e não substitui a avaliação de um especialista. Seu propósito é reunir as evidências encontradas, explicar por que uma consulta recebeu determinado nível de atenção e auxiliar na decisão sobre quais mudanças devem ser testadas.

2 MATERIAIS E MÉTODOS

Iniciou-se a parte prática com a construção de um protótipo capaz de executar consultas SQL, coletar informações sobre essas execuções e apresentar possíveis pontos de atenção. O estudo foi realizado em um ambiente controlado, com dados fictícios, para que os testes não interferissem em um sistema real.

O trabalho foi organizado em seis etapas. Primeiro, construiu-se o banco experimental. Depois, geraram-se e carregaram-se os dados. Na terceira etapa, executou-se um conjunto de consultas para formar um histórico. Na quarta, coletaram-se as estatísticas e selecionaram-se as consultas mais relevantes. Na quinta, analisaram-se os planos de execução. Por fim, comparou-se uma alteração de consulta e verificou-se seu resultado. A Figura 1 apresenta esse processo.

Figura 1 - Etapas da solução desenvolvida



Fonte: elaboração própria.

2.1 AMBIENTE DE DESENVOLVIMENTO

Na primeira rodada experimental foram utilizados PostgreSQL 18.1, Python 3.14.3 e Psycopg 3.3.6 em Windows de 64 bits. O PostgreSQL armazenou os dados, executou as consultas e forneceu seus planos. O Python foi usado para gerar os dados, organizar os testes e apresentar os resultados. O Psycopg fez a comunicação entre o programa e o banco. O DBeaver auxiliou na administração e na conferência das tabelas.

Após a formatação do computador, o projeto foi preparado novamente com PostgreSQL 18.6 e Python 3.14.7. Nessa verificação, o computador possuía processador AMD Ryzen 5 5600X, aproximadamente 32 GB de memória RAM, SSD de 2 TB e Windows 11 Pro. Os resultados da primeira rodada foram mantidos separados dessa verificação posterior, pois mudanças no ambiente podem influenciar os tempos.

Os arquivos foram organizados em pastas para os scripts SQL, os programas Python, a documentação e os resultados. Também foi criado um ambiente virtual Python para registrar as dependências do projeto e facilitar uma nova instalação.

2.2 BANCO EXPERIMENTAL E GERAÇÃO DOS DADOS

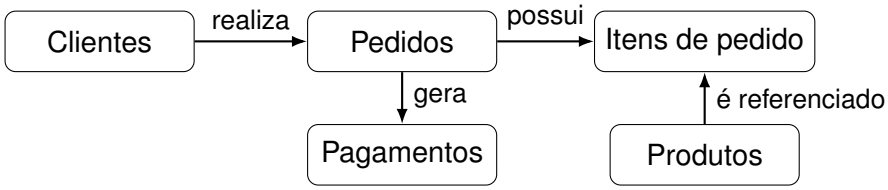
Criou-se o banco `tcc_sql_optimizer` com cinco tabelas: clientes, produtos, pedidos, itens de pedido e pagamentos. Um cliente pode realizar vários pedidos; cada pedido pode possuir vários itens; cada item associa um produto ao pedido; e o pagamento registra a situação financeira do pedido. Essa estrutura representa operações comuns de uma loja

e permite testar filtros, junções, agrupamentos e ordenações.

Todas as tabelas receberam chave primária. Também foram definidas chaves estrangeiras para preservar os relacionamentos. O e-mail do cliente foi definido como único, criando um índice que depois pôde ser utilizado no teste de busca por e-mail. Não foram acrescentados índices apenas para favorecer as consultas do experimento.

A Figura 2 resume os relacionamentos. Ela também ajuda a compreender por que algumas consultas precisam juntar várias tabelas: para chegar dos dados de um cliente aos produtos comprados, por exemplo, o banco passa pelos pedidos e pelos itens.

Figura 2 - Relacionamentos principais do banco experimental



Fonte: elaboração própria.

O programa de geração criou os volumes apresentados na Tabela 1. As quantidades não vieram de uma empresa real. Elas foram escolhidas para combinar tabelas pequenas e grandes, produzir consultas rápidas e lentas e tornar visível a diferença entre uma leitura direcionada e uma leitura de muitos registros.

Tabela 1 - Volume do banco experimental

Tabela	**Quantidade de registros**
Clientes	100.000
Produtos	50.000
Pedidos	1.000.000
Itens de pedido	2.498.714
Pagamentos	1.000.000
Total	**4.648.714**

Fonte: elaboração própria.

Os dados incluíram diferentes estados, preços, datas, situações de pedido e frequências de compra. A distribuição foi controlada para gerar casos variados, mas não buscou reproduzir o comportamento de uma empresa específica. Depois da carga, utilizou-se o comando `ANALYZE` para

que o PostgreSQL atualizasse as informações usadas na criação dos planos.

2.3 CONSULTAS E EXECUÇÃO DA CARGA

Foram elaboradas 12 consultas SQL. Elas receberam nomes de Q01 a Q12 apenas para facilitar a organização do estudo. O conjunto incluiu filtros simples, junções, agrupamentos, ordenações, subconsultas e consultas que retornam muitos registros. A Tabela 2 resume a carga executada.

Tabela 2 - Consultas utilizadas no experimento

Consulta	Chamadas	Objetivo principal
Q01	500	Buscar um cliente pela chave primária.
Q02	100	Filtrar e ordenar produtos.
Q03	60	Consultar pedidos de um cliente.
Q04	20	Listar pedidos cancelados.
Q05	15	Buscar pedidos de um ano e os respectivos clientes.
Q06	10	Agrupar dados das cinco tabelas.
Q07	8	Somar pedidos por mês.
Q08	6	Ordenar pedidos de maior valor.
Q09	5	Buscar cliente aplicando uma função ao e-mail.
Q10	3	Buscar pedidos acima do valor médio.
Q11	2	Contar pedidos por cliente.
Q12	1	Ler todos os itens de pedido.
Total	**730**	**Chamadas executadas.**

Fonte: elaboração própria.

As quantidades foram definidas para criar três situações fáceis de comparar: uma consulta muito frequente, consultas intermediárias e consultas pouco frequentes que processam muitos dados. Antes da carga completa, executou-se cada consulta uma vez para verificar seu funcionamento. Em seguida, realizaram-se as 730 chamadas com valores de entrada fixos. Não foram simulados vários usuários ao mesmo tempo.

2.4 COLETA E CLASSIFICAÇÃO DAS CONSULTAS

A coleta utilizou a extensão `pg_stat_statements`, que mantém estatísticas acumuladas das consultas executadas (PostgreSQL Global Development Group, 2026a). Para cada consulta, foram observados principalmente o número de chamadas, o tempo total, o tempo médio e a quantidade de linhas retornadas.

Essas medidas respondem a perguntas diferentes. O tempo to-

tal mostra onde o banco gastou mais tempo no conjunto da carga. O tempo médio mostra quanto uma chamada costuma demorar. A frequência mostra quais consultas foram executadas mais vezes. Por isso, uma consulta frequente não é necessariamente a mais lenta, e uma consulta executada apenas uma vez pode retornar muitos dados.

A Tabela 3 apresenta uma leitura resumida dessas informações. Não há uma fórmula de classificação que determine sozinha qual consulta deve ser alterada. As métricas funcionam como pontos de partida para selecionar o que será investigado.

Tabela 3 - Significado das principais métricas

Métrica	**O que informa**	**Como foi utilizada**
Chamadas	Quantas vezes a consulta apareceu.	Identificar consultas frequentes.
Tempo total	Tempo somado de todas as chamadas.	Localizar onde o banco acumulou mais trabalho.
Tempo médio	Duração média de uma chamada.	Encontrar consultas individualmente demoradas.
Linhas	Quantidade de registros retornados.	Evitar comparar consultas de volumes muito diferentes como se fossem iguais.

Fonte: elaboração própria.

O coletor relacionou as estatísticas com as consultas Q01 a Q12 e exportou os resultados para arquivos CSV e JSON. Esses arquivos facilitaram a conferência e permitiram que a interface apresentasse as informações sem exigir a leitura direta das tabelas internas do PostgreSQL.

2.5 ANÁLISE PREVENTIVA DOS PLANOS DE EXECUÇÃO

O analisador recebe uma instrução `SELECT` e solicita ao PostgreSQL apenas o plano estimado, por meio de `EXPLAIN` sem `ANALYZE`. Assim, a consulta candidata não é executada durante essa análise (PostgreSQL Global Development Group, 2026b). O plano mostra como o banco pretende localizar, combinar e ordenar os dados.

O protótipo procura sinais simples que merecem atenção: uso de `SELECT *`, leitura sequencial de uma tabela grande, função aplicada sobre uma coluna usada no filtro, retorno de muitas linhas e diferença elevada entre a quantidade prevista no plano e a quantidade observada no histórico.

Cada sinal gera uma explicação e uma sugestão de investigação.

Os níveis baixo, médio e alto servem somente para ordenar os casos. Eles não provam que uma consulta está errada e não representam tempo de execução. A ferramenta também não cria índices nem altera comandos automaticamente. A decisão depende da análise do desenvolvedor e de um teste posterior.

2.6 COMPARAÇÃO E VALIDAÇÃO

Selecionaram-se dois casos para demonstração. Na consulta de pedidos por ano, comparou-se o plano da forma original, que aplicava uma função sobre a data, com o plano de uma alternativa que utilizava um intervalo de datas. Essa comparação foi apenas preventiva: as consultas não foram executadas para medir qual delas seria mais rápida.

Na busca por e-mail, comparou-se a aplicação de `LOWER` sobre a coluna com uma comparação direta. A alternativa poderia usar o índice já existente. Como todos os e-mails da base foram armazenados em letras minúsculas, verificou-se se as duas formas retornavam o mesmo registro. Depois, cada versão foi executada 100 vezes. O programa comparou os tempos médios informados pelo PostgreSQL. O percentual apresentado nos resultados foi calculado automaticamente a partir da diferença entre essas duas médias, sem exigir uma fórmula específica do usuário da ferramenta.

2.7 INTERFACE DO PROTÓTIPO

Desenvolveu-se uma interface web local para facilitar a apresentação do projeto. A tela inicial resume a quantidade de consultas, o número de chamadas e o tempo acumulado. Outras telas permitem escolher uma consulta, visualizar seu histórico, analisar o plano estimado e comparar a versão original com uma alternativa.

A interface informa quando uma operação é apenas preventiva e quando houve uma medição real. Também impede comandos de alteração, como `INSERT`, `UPDATE` e `DELETE`, pois o objetivo do protótipo é analisar instruções de leitura. Foram realizados testes de funcionamento das principais rotas e da comunicação com o banco.

3 RESULTADOS E DISCUSSÃO

Os resultados foram organizados de acordo com as etapas descritas na metodologia. Primeiro, apresentam-se as estatísticas da carga.

Depois, discutem-se duas consultas selecionadas e a interface criada para tornar a análise mais visual.

3.1 RESULTADOS DA CARGA DE CONSULTAS

O teste inicial executou as 12 consultas sem erros. Na carga completa, foram realizadas 730 chamadas. O programa Python registrou aproximadamente 103,2 segundos para essa rodada. O valor inclui o tempo necessário para o programa receber os resultados, enquanto as estatísticas internas do PostgreSQL registraram aproximadamente 102,6 segundos de execução no servidor.

Os três critérios de classificação destacaram consultas diferentes, como mostra a Tabela 4.

Tabela 4 - Consultas destacadas pelas estatísticas

Consulta	Chamadas	Total (ms)	Média (ms)	Motivo do destaque
Q05	15	42.472,08	2.831,47	Maior tempo acumulado.
Q12	1	6.795,63	6.795,63	Maior duração em uma chamada.
Q01	500	9,25	0,02	Maior frequência.

Fonte: elaboração própria.

A Q05 foi o principal caso para investigação porque concentrou o maior tempo total. A Q12 apresentou o maior tempo por chamada, mas foi executada apenas uma vez e retornou 2.498.714 linhas. Portanto, esse valor não deve ser tratado como uma média representativa. A Q01 foi executada 500 vezes, porém continuou rápida por buscar um cliente pela chave primária. Esses resultados mostram por que frequência, tempo e volume precisam ser observados em conjunto.

3.2 CONSULTA DE PEDIDOS POR ANO

A Q05 seleciona pedidos de 2026 e os dados dos respectivos clientes. Na forma original, o filtro extrai o ano de cada data:

```
SELECT *
FROM public.pedidos p
JOIN public.clientes c ON c.id = p.cliente_id
WHERE EXTRACT(YEAR FROM p.data_pedido) = 2026;
```

O plano estimou 5.000 linhas, enquanto o histórico indicou uma média próxima de 499.676 linhas por execução. Em outras palavras, a consulta normalmente retornou quase 100 vezes mais linhas do que o plano previa. O analisador também identificou a seleção de todas as colunas, a leitura sequencial da tabela de pedidos e a função aplicada sobre a data.

Preparou-se uma alternativa que compara a coluna diretamente com o início e o fim do ano:

```
WHERE p.data_pedido >= TIMESTAMP '2026-01-01 00:00:00'
  AND p.data_pedido <  TIMESTAMP '2027-01-01 00:00:00'
```

No novo plano, a previsão passou para aproximadamente 499.434 linhas, valor muito mais próximo do histórico. Isso mostra que a escrita do filtro ajudou o banco a prever melhor o volume de saída. Entretanto, não se afirmou que a alternativa ficou mais rápida, pois ela não foi executada nessa comparação. O resultado foi mantido como uma recomendação para teste posterior.

Tabela 5 - Resumo da comparação da consulta de pedidos por ano

Aspecto	**Forma original**	**Alternativa**
Filtro	Extração do ano da data.	Comparação com início e fim do ano.
Linhas previstas	5.000	Aproximadamente 499.434
Referência histórica	Aproximadamente 499.676 linhas por chamada.	
Conclusão	Previsão distante do histórico.	Previsão mais próxima; tempo não medido.

Fonte: elaboração própria.

3.3 BUSCA POR E-MAIL

A Q09 aplicava a função `LOWER` sobre a coluna de e-mail. Mesmo existindo um índice nessa coluna, o plano original previa uma leitura sequencial dos 100.000 clientes. A alternativa comparou o e-mail diretamente com o valor já normalizado em letras minúsculas, permitindo a utilização do índice.

```
-- Forma original
WHERE LOWER(email) = 'cliente123@example.com';

-- Alternativa avaliada
WHERE email = 'cliente123@example.com';
```

As duas formas retornaram o mesmo cliente nos testes. Em 100 chamadas de cada versão, o tempo médio informado pelo servidor caiu de 26,33 milissegundos para 0,0303 milissegundo. Isso correspondeu a uma redução aproximada de 99,88% nas condições do experimento. O resultado não significa que toda função deva ser removida de um filtro. Nesse caso específico, a mudança foi válida porque os e-mails da base já estavam armazenados em letras minúsculas.

Tabela 6 - Resumo da comparação da busca por e-mail

Aspecto	**Forma original**	**Alternativa**
Comparação	Função `LOWER` sobre a coluna.	Comparação direta com o e-mail.
Plano	Leitura sequencial dos clientes.	Uso do índice existente.
Tempo médio	26,33 ms	0,0303 ms
Resultado	Mesmo cliente retornado nos testes realizados.	

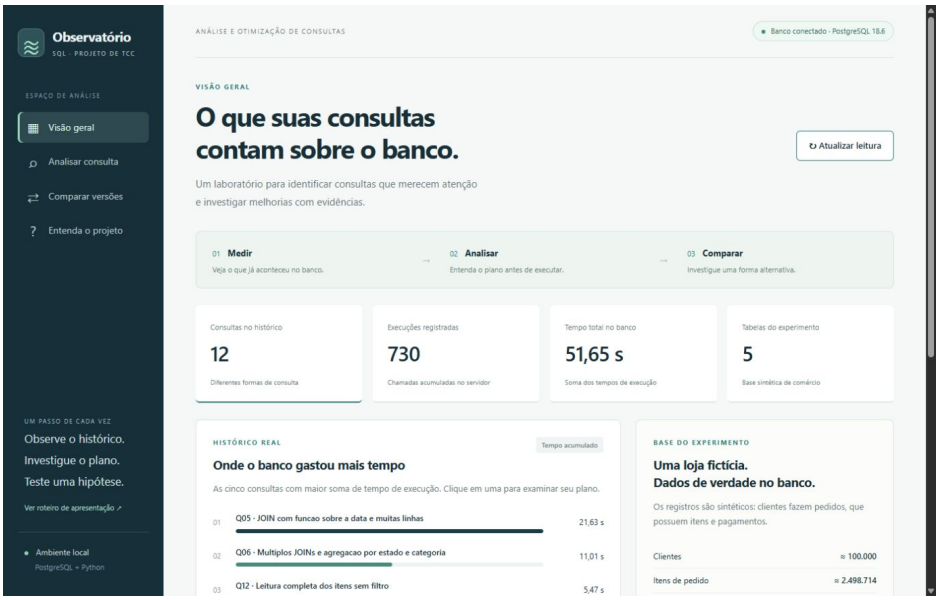
Fonte: elaboração própria.

A comparação também mostrou uma diferença importante entre previsão e medição. Para a consulta por ano, houve somente análise do plano. Para a busca por e-mail, houve comparação do resultado e execução repetida das duas versões. Somente o segundo caso permitiu apresentar uma redução de tempo medida.

3.4 APRESENTAÇÃO VISUAL

A interface reuniu o histórico, os planos e as sugestões em uma apresentação mais simples. A Figura 3 mostra a visão geral, que apresenta as quantidades do experimento e as consultas com maior tempo acumulado.

Figura 3 - Visão geral da interface web



Fonte: elaboração própria.

Na tela de análise, o usuário pode escolher um exemplo ou colar uma instrução `SELECT`. O sistema mostra o plano estimado, os sinais encontrados e uma explicação de cada sugestão. Na tela de comparação, a versão original e a alternativa são apresentadas lado a lado. Essa organização permite demonstrar o funcionamento do projeto sem depender apenas do terminal.

Uma verificação posterior confirmou a carga com 730 chamadas e a leitura das 12 consultas pela interface. Também foram testados a rejeição de SQL inválido, o caso de uma consulta sem histórico e a análise sem execução da consulta candidata. Esses testes verificaram o funcionamento do protótipo, mas não correspondem a uma avaliação de usabilidade com participantes.

3.5 LIMITAÇÕES DOS RESULTADOS

O estudo utilizou dados fictícios, valores de filtro fixos e quantidades de chamadas definidas para o experimento. Por isso, os resultados não permitem afirmar que a mesma classificação ocorreria em uma empresa real. Além disso, os tempos podem variar conforme o computador, o uso de memória e outros programas em execução.

A Q12 teve somente uma chamada e, portanto, seu tempo não representa várias observações. A consulta por ano teve apenas uma comparação preventiva de planos. Já o ganho da busca por e-mail depende de

os endereços estarem armazenados em letras minúsculas. Essas condições precisam ser mencionadas ao apresentar os resultados.

O protótipo foi desenvolvido para PostgreSQL e uso local. Sua aplicação em produção exigiria novos testes, controle de permissões e avaliação com diferentes consultas e volumes de dados.

4 CONCLUSÃO

Este trabalho desenvolveu um processo para identificar e analisar consultas SQL com possíveis problemas de desempenho. A solução combinou estatísticas históricas do PostgreSQL, planos de execução, regras de análise e uma interface web.

Os resultados mostraram que tempo total, tempo médio e frequência destacam situações diferentes. A consulta de pedidos por ano apresentou o maior tempo acumulado, a leitura de todos os itens teve a maior duração em uma única chamada e a busca pela chave primária foi a mais frequente. Assim, uma consulta não deve ser escolhida para análise com base em apenas um número.

Na consulta de pedidos por ano, a alteração do filtro aproximou a quantidade prevista da quantidade observada no histórico, mas não foi medida como otimização. Na busca por e-mail, a comparação direta permitiu o uso do índice existente e reduziu o tempo médio do servidor de 26,33 para 0,0303 milissegundo, mantendo o mesmo resultado nas condições avaliadas.

Concluiu-se que a combinação de histórico e planos pode ajudar a localizar consultas relevantes e explicar por que uma alteração merece ser testada. A ferramenta não modifica automaticamente o banco e não substitui a validação. Sua função é organizar as evidências e apoiar a decisão do desenvolvedor.

Como continuidade, podem ser realizados novos testes com diferentes valores de entrada, mais repetições e outros bancos de dados. Também pode ser feita uma avaliação da interface com usuários para verificar se as explicações são suficientemente claras.

REFERÊNCIAS

KOSSMANN, J.; PAPENBROCK, T.; NAUMANN, F. Data dependencies for query optimization: A survey. **The VLDB Journal**, v. 31, n. 1, p. 1–22, 2022. Disponível em: <https://doi.org/10.1007/s00778-021-00676-3>. Acesso em: 12 out. 2026.

LAN, H.; BAO, Z.; PENG, Y. A survey on advancing the dbms query optimizer: Cardinality estimation, cost model, and plan enumeration. **Data Science and Engineering**, v. 6, n. 1, p. 86–101, 2021. Disponível em: <https://doi.org/10.1007/s41019-020-00149-7>. Acesso em: 12 out. 2026.

PostgreSQL Global Development Group. **PostgreSQL 18 Documentation: pg_stat_statements**. [S.l.], 2026. Disponível em: <https://www.postgresql.org/docs/18/pgstatstatements.html>. Acesso em: 12 out. 2026.

PostgreSQL Global Development Group. **PostgreSQL 18 Documentation: Using EXPLAIN**. [S.l.], 2026. Disponível em: <https://www.postgresql.org/docs/18/using-explain.html>. Acesso em: 12 out. 2026.